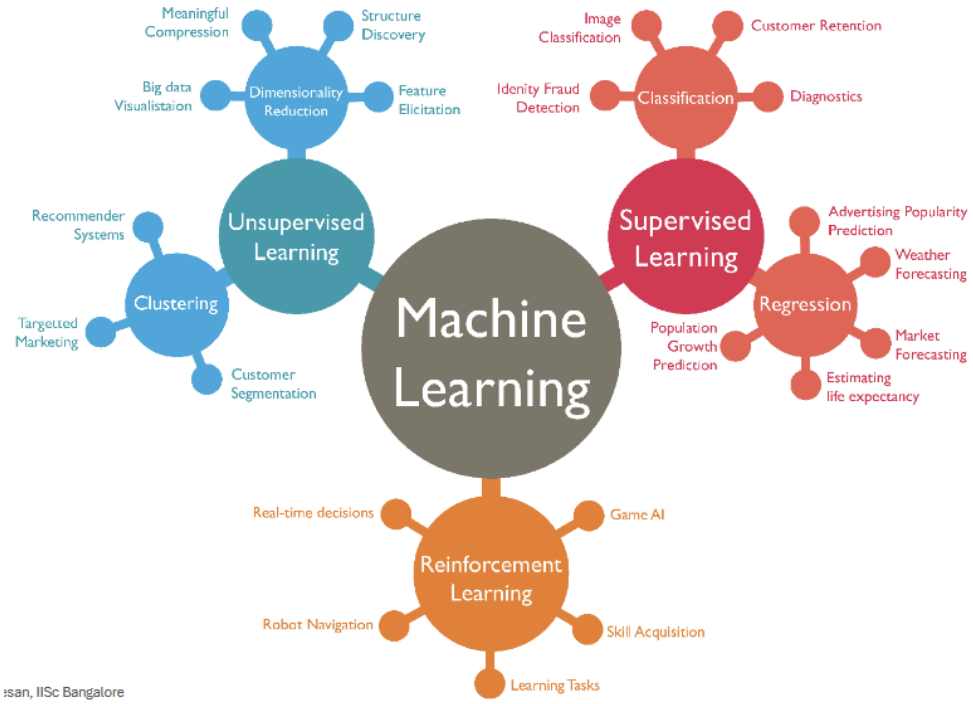
Rishabh Indoria
SR NO: 24002
M.Tech AI

Generative and Agentic AI

March 11, 2026

1 Foundations

- **Machine Learning**: Ability of a system to make predictions without being explicitly programmed. Developing and implementing algorithms to receive data and process it to analyze and learn patterns, insights, relations, etc.



- AI is a field in interdisciplinary science that focuses on creating systems/agents capable of performing tasks that typically require human intelligence.
- **Narrow AI**: Designed to perform a narrow task (e.g., facial recognition, internet searches, or driving a car). Most current AI applications fall under this category.
- **General AI or AGI**: Refers to systems that possess the ability to understand, learn, and apply intelligence in a wide range of contexts, much like a human being.
- **Artificial Super Intelligence (ASI)**: A type of AI that surpasses human intelligence and can perform any task better than a human.
- **Key characteristics of AI Systems**
  1. Ability to learn from data, adapt to changes, and execute tasks autonomously.
  2. AI relies on vast datasets, efficient algorithms, and computational resources.
  3. AI can simulate human reasoning, analyze patterns, and solve complex problems.

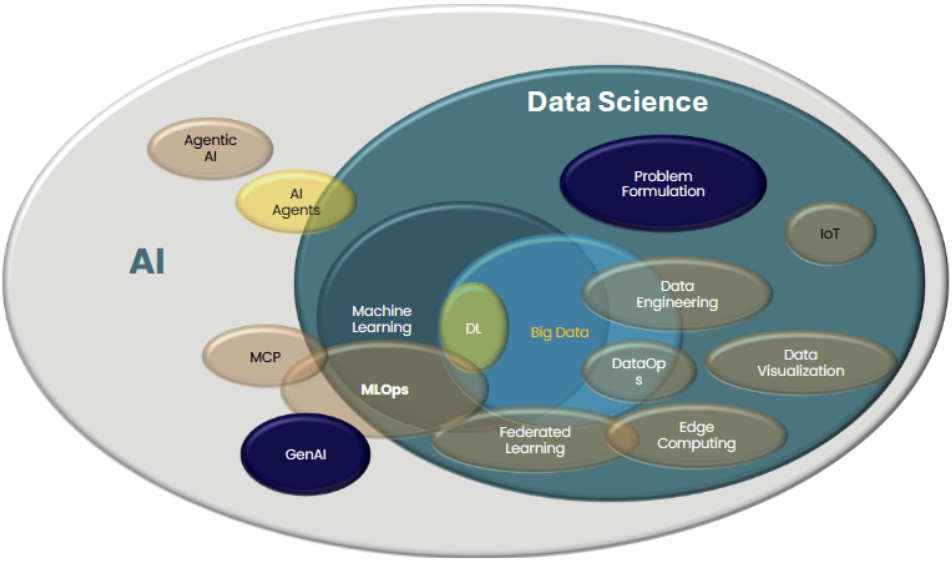


Figure 1: AI, ML, and Data Science

- **Generative AI** refers to artificial intelligence systems that create new content, such as text, images, or audio, based on learned patterns from existing data. Core characteristics include
  1. **Content Creation**: Produces new data instances.
  2. **Pattern Learning**: Captures underlying data structures.
  3. **Diverse Outputs**: Generates varied outputs that resemble training data.
  4. **Unsupervised Learning**: Operates without explicit labels.
  5. **Creativity**: Capable of producing unexpected and innovative results.
- **Predictive AI** focuses on forecasting based on past data. Outputs are probabilistic predictions.
- **AI model development** is an iterative and cyclical process, not a one-time event.
  1. It begins with **Project Scoping** to define the problem and objectives.
  2. **Data Engineering** is a crucial foundational step, involving data collection, cleaning, and preparation.
  3. **ML Model Development** is where the core ML algorithms are trained.
  4. **Deployment** integrates the AI model into a production environment for real-world use.
  5. **Monitoring & Continual Learning** is vital for tracking AI model performance, identifying issues, and enabling ongoing improvements.
  6. **Finetuning** based on real-world feedback, leading back to potentially re-scoping or further data engineering.
- **Why AI Fails**
  1. Fragmented AI efforts lead to slow development, security risks, and competitive disadvantage.
  2. Corporate AI Projects Are Failing to Scale, upto 80% of AI projects fail, which is roughly double the failure rate of traditional IT initiatives.
- **Domain Expert**

Figure 2: Domain Expert Skills

- **Data Engineer**
  1. Transform data into a useful format for analysis.
  2. Creating software solutions around big data, creating data pipelines.
  3. Understand the right tool.

Figure 3: Data Engineer

- **Data Scientist**
  1. Data analyst who applies ML/AI.
  2. Interact with the business side, understand the domain enough to make insights.

3. Verbally and visually communicate complex results in a way that the business can understand and act on them.

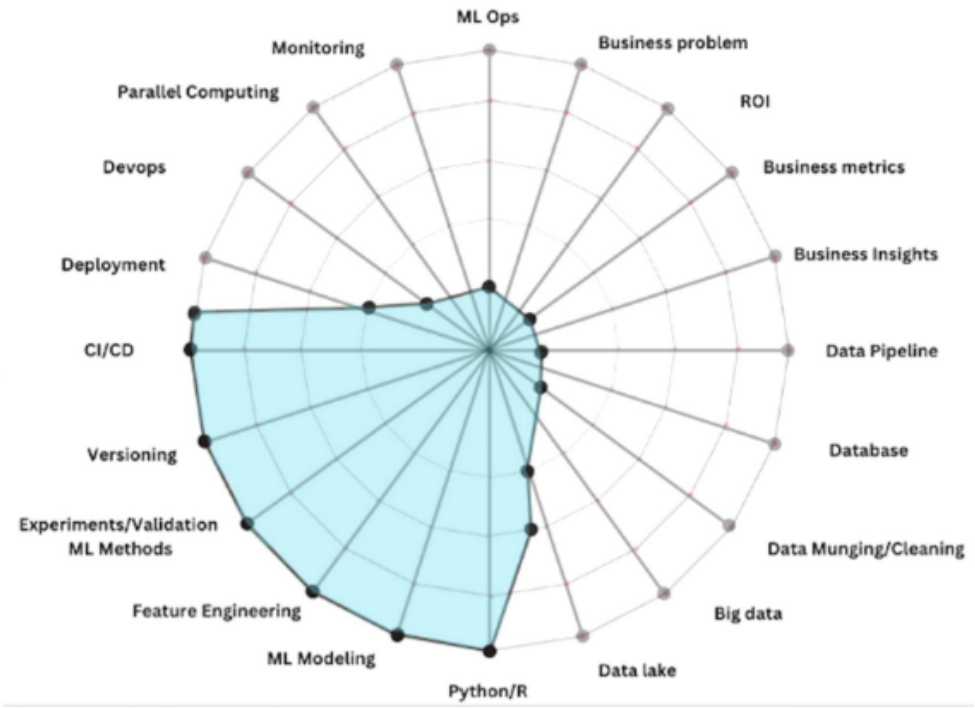


Figure 4: Data Scientist

- **ML Engineer**
  1. Needs to apply DevOps best practices.
  2. Cross-trained enough to become proficient at both data engineering and data science.
  3. **Cloud Native ML Platform**: AWS SageMaker, Azure ML Studio, GCP AI Platform
  4. **Containerized Workflow**: Docker, Kubernetes, public and private containers.
  5. **Serverless technology**: AWS Lambda, AWS Athena, Google Cloud Foundation, Azure Functions
  6. **Speacial hardware for ML**: GPUs, TPUs, A14, AWS Inferentia Elastic Inference
  7. **Big data platforms**: Databricks, Hadoop/Spark, Snowflake, Amazon EMR, Google BigQuery.

Figure 5: ML Engineer

- An AI Team would involve some combination of the above 4 careers.

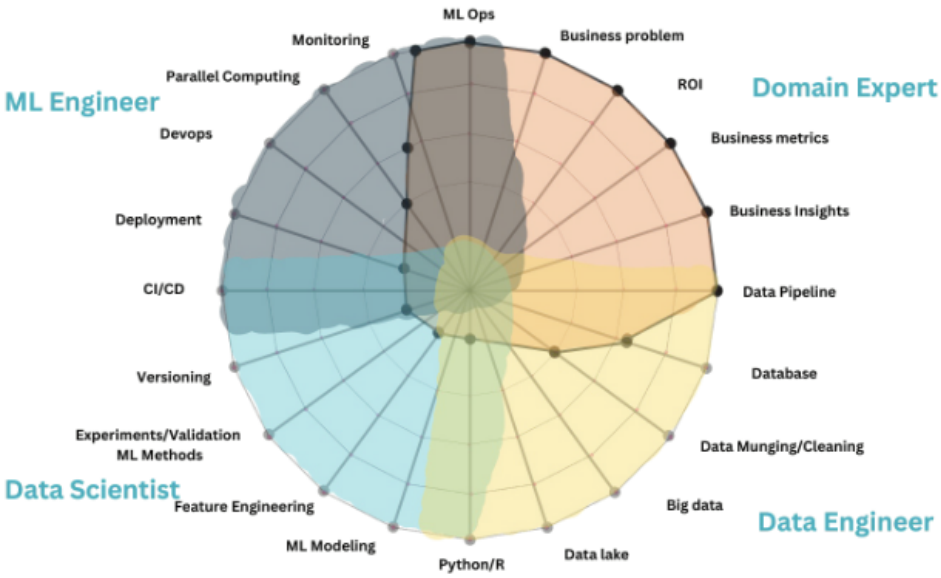


Figure 6: AI Team

2 Prompting Principles

- A methodological approach to design effective input texts (prompts) that directly influence how LLMs understand, process, and respond to user prompts with accurate and contextually appropriate outputs.
- Five-Level Framework for Working with LLMs
  1. **Basic Prompting (Conversational Level)**: Users interact with LLMs through chatbot interfaces (ChatGPT, Claude, etc.) by writing natural language prompts.
     Characteristics: No coding required, users create effective prompt through trial and error, limited control over model behavior.
     Limitations: Inconsistent results, no systematic optimization, limited reproducibility.
  2. **Prompt Engineering (API Level)**: Systematic design of prompt templates with API calls, controlling parameters like `temperature` and `top_p` for consistent, optimized outputs.
     Key Parameters: Temperature$(0.0 - 2.0)$ controls randomness/creativity, Top_p$(0.0 - 1.0)$ is nucleus sampling for output diversity, Max_Tokens for Output length control, and System prompts for role and behavior definition.
     Advantages: Reproducible, programmable, batch processing capable.
  3. **RAG (Retrieval-Augmented Generation)**: Enhances LLM responses by retrieving relevant information from external knowledge bases and augmenting the prompt with contextual data.
     Advantages: reduces hallucinations, Provides up-to-date information, Enables domain-specific responses, Source citation capability.
  4. **Fine-Tuning (Parameter-Efficient Methods)**: Adpating pre-trained models to specific tasks by trainined additional parameters while keeping the base model frozen.
     Popular Methods: LoRA/Low-Rank Adaptation(Adds trainable low-rank matrices, $\sim 0.1 - 1\%$ of original parameters), QLoRA(Qunatized version using 4-bit precision for memory efficiency), DoRA(Weight decomposed adaptation for better performance), Adapters(Small trainable modules inserted between layers).
  5. **Training LLMs from Scratch**: Building and training a new language model from random initialization on custom datasets.
     When to Consider: Highly specialized domains with unique vocabulary, proprietary architectures needed, complete control over model behavior required, security/privacy constraints prohibiting external models.
- **The Anatomy of Effective Prompt**
  1. **Structure**: The structure of a prompt refers to how it is organized to guide the AI's understanding and response generation. The prompt should provide a well-defined starting point, outline the task and

desired outcome. Structure the prompt in a way that guides the AI step-by-step, ensuring a coherent thought process.

2. **Clarity**: The language used in the prompt should be unambiguous and easy for the AI to comprehend. Avoid complex jargon or overly technical terms. Use words that have clear and defined meaning in the context of the task
3. **Precision**: Precision in the prompt is crucial for achieving the desired level of detail and accuracy in the AI's response. It involves using specific terms and details that tailor the AI's output to your exact needs. The more information you provide, the better the AI can tailor its response to your needs.

2.1 Core Prompting Techniques

- **Zero-Shot Chain-of-Thought (CoT) Prompting**: Step-by-Step Reasoning, Breaking down complex problems, transparency of reasoning
  Without CoT Prompt: "What is 28×47?"
  With CoT Prompt: "What is 28×47? Show your step-by-step calculation."
  Applications: Mathematical word problems, Logic puzzles and riddles, Multi-step instructions, common sense reasoning.
  Key Limitations: Domain expertise required, inconsistent with very simple tasks, language dependent, not suitable for creative tasks.
- **Few-Shot Chain-of-Thought (CoT) Prompting**: Probiding Reasoning Demonstrations
  Few-Shot Without Reasoning:
  1. Example 1: "Roger has 5 tennis balls. He buys 2 more cans of 3 balls each. Total: 11 balls."
  2. Example 2: "A cafeteria had 23 apples. They used 20 for lunch. Remaining: 3 apples."
  3. Query: "Jack has 15 marbles. He loses 7, then finds 4. How many does he have?"
  4. Response: "12 marbles"
  5. Problem: Examples show only answers, not the reasoning process.

  Few-Shot With Reasoning:
  1. Example 1: "Roger has 5 tennis balls. He buys 2 more cans of 3 balls each. Let me calculate: $5 + (2 \times 3) = 5 + 6 = 11$ balls."
  2. Example 2: "A cafeteria had 23 apples. They used 20 for lunch. Let me calculate: $23 - 20 = 3$ apples remaining."
  3. Query: "Jack has 15 marbles. He loses 7, then finds 4. How many does he have?"
- **ReAct (Reasoning + Acting) Framework**: Though-Action-Observation Cycles, Integration with External tools, Dynamic problem solving.
- **Sequential Chaining**: Linear Prompt sequences
- **Conditional Chaining**: Decision-based routing, all inputs follow analyze $\rightarrow$ categorize $\rightarrow$ respond
- **Parallel Chaining**: Concurrent Prompt Execution

3 LLM Foundations

- A good reference is `https://transformer-circuits.pub/2021/framework/index.html`
- **Tokenization**: The process of splitting up the sentences into individual words or subwords (tokens).
- **Subword Tokenization**: A middle ground between word and character-based tokenization strategies. Frequently used words should not be split into smaller subwords. Rare words should be decomposed into meaningful subwords.

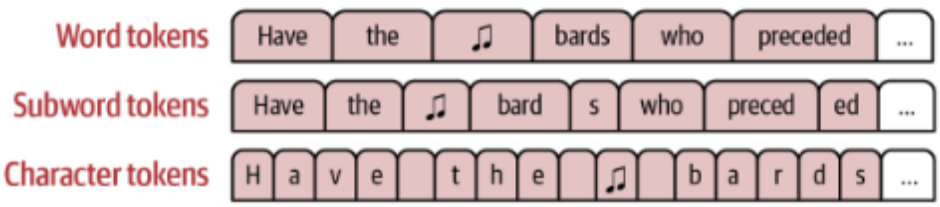


Figure 7: Subword Tokenization

- **The Tokenization Process**: Aims to create an optimal vocabulary of subwords from a large text corpus.
  1. **Initial Vocabulary Creation**: The process begins by creating a large initial vocabulary. This can include all characters, common substrings, or even the result of a preliminary tokenization with another algorithm like BPE.
  2. **Loss Calculation**: The algorithm then calculates a "loss" for the current vocabulary. This loss is typically the negative log-likelihood of the training corpus given the vocabulary. In simpler terms, it measures how well the current set of tokens can represent the text.
  3. **Token Removal**: The model then iteratively removes a certain percentage of tokens from the vocabulary. The tokens that are removes are those that cause the smallest increase in the overall loss. This means the algorithm prunes the tokens that are least important for representing the corpus.
  4. **Repitition**: These steps of loss calculation and token removal are repeated until the vocabulary reaches a predefined target size.

Feature	Unigram Tokenization	Byte-Pair Encoding (BPE)	WordPiece
Approach	Starts with a large vocabulary and removes less likely tokens.	Starts with individual characters and merges the most frequent pairs.	Similar to BPE, but merges pairs that maximize the likelihood of the training data.
Nature	Probabilistic; can produce multiple segmentations with different probabilities.	Deterministic; always produces the same segmentation for a given word.	Deterministic; always produces the same segmentation for a given word.
Flexibility	More flexible and can better model the morphology of a language.	Less flexible as it is based on frequency counts.	Less flexible than Unigram but considers likelihood.

Figure 8: Tokenization

- **Embedding**: In any language processing task, we start with raw text data. However, transformers work only with numerical data. It is therefore necessary to convert text tokens into continuous numerical representation, called embeddings. Input embedding layer maps discrete token indices (integers) to dense vectors of fixed dimensions, which capture semantic meanings and provide a compact representation of the input for further processing.
- **Positional Encoding**: Transformer processes all the elements of a sequence in parallel without any regard for their order (self-attention is permutation invariant). Explicitly add positional information to indicate where a word appears in a sequence. Common approaches include binary, integer, sinusoidal, and rotary.
- **Encoder-only Models**: Convert an input sequence of text into a rich numerical representation that is well-suited for tasks like text classification or named entity recognition. The representation computed for a given token in this architecture depends both on the left and the right contexts. This is often called **bidirectional attention**.
- One of the challenges in FM-building is the lack of task-specific training data. User BERT(Bidirectional Encoder Representations) trained using MLM (Masked Language Modeling, Left) and NSP (Next Sentence Prediction, Right) objectives.
- **Decoder-only Models**: Given a prompt of text like "I like LLMs, I had a ..." these models will auto-complete the sequence by iteratively predicting the most probable next word. The family of GPT models belongs to this class. The representation computed for a given token in this architecture depends only on the left context. This is often called **causal** or **autoregressive attention**.
- Can we leverage a large corpus of unlabeled data to pretrain an FM that understands general patterns? GPT(Generative Pretrained Transformer) was trained to predict the next token, given the previous tokens (more learning signals than MLM).
- **Encoder-Decoder Models**: These are used for modeling complex mappings from one sequence of text to another. Suitable for machine translation and summarization tasks. BART and T5 models belong to this class.
- Some good videos to understand Attention Mechanisms
  - `https://youtu.be/wjZofJX0v4M?feature=shared`

- https://youtu.be/eMlx5fFNoYc?feature=shared

- Core Dimensions in Attention
  1. N (Context Length): Number of tokens/words in the sequence
  2. d_{model}: Dimension of input embeddings
  3. d_k, d_q: Dimension of Key and Query vectors (usually equal)
  4. d_v: Dimension of Value vectors

- **Query (Q)**: "What am I looking for?". Represents what information each token wants to retrieve, like a search query in a database.
  W_Q: Learn what to look for, $Q = X \cdot W_Q$

- **Key (K)**: "What information do I offer?". Represents what information each token can provide to others, like metadata/index terms in a database.
  W_K: Learn what to advertise, $K = X \cdot W_K$

- **Value (V)**: "What is my actual content?". The actual information that will be aggregated, like the actual records in a database.
  W_V: Learn what to provide, $V = X \cdot W_V$

- Computing Attention Scores
  $$\text{Scores} = \frac{Q \cdot K^T}{\sqrt{d_k}}$$
  Apply row-wise softmax to get attention weights (A).
  Final Output $= A \cdot V$

- **GPT-3 Scale example**
  1. Context Length (N): 2048 tokens
  2. d_{model}: $12,288$
  3. d_k, d_v: 128 (per head)
  4. Number of heads: 96
  5. Attention matrix per head: $2048 \times 2048 = 4.2M$ values
  6. Total attention across all heads: $4.2M \times 96 = 403M$ per layer.

- Memory: $O(N^2)$, quadratic in sequence length

- Computation: $O(N^2 \times d)$, quadratic in sequence, linear in dimension.

- This is why context length is limited in transformers.

- **Masked Attention**: In a crude self-attention language model, there would be several alternating self-attention layers and position-wise feedforward networks. Self-attention at step 1 can look at the value at steps 2 and 3, which is a big problem. At test time, the inputs at steps 2 and 3 will be based on the output at step 1, which requires knowing the input at steps 2 and 3. Must allow self-attention into the past, but not into the future,
  $$e_{l,t} = \begin{cases} q_l \cdot k_t & \text{if } l \geq t \\ -\infty & \text{otherwise} \end{cases}$$
  In practice, just replace $exp(e_{l,t})$ with 0 if $l < t$.

- **Cross-Attention**: Acts as a bridge between the encoder and decoder. Compares the query from the decoder with the keys from the encoder. It calculates how well each query matches each key. The formula is very similar to self-attention, except the query (Q) comes from the decoder's output.

- **Efficient Attention Mechanisms**: Dark cells mean, $A(i,j) \neq 0$ (i.e., i attends to j), and light cells mean $A(i,j) = 0$(i.e., i does not attend to j).

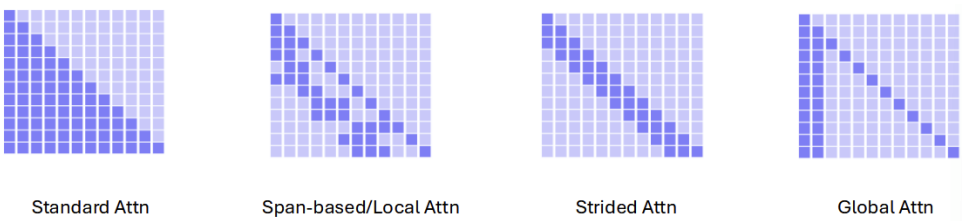


Figure 9: Efficient Attention Mechanisms

4 RAG & Vector Databases

- **Token Embeddings**: Token embeddings are vector representations of individual words or subwords. They convert discrete text units into dense, continuous vectors that a computer can process.
  Examples: Word2Vec, GloVe, BERT token embeddings
- **Sentence Embeddings**: Single, holistic vector representations of an entire sentence, paragraph, or even a whole document. Aim to capture the collective semantic meaning of the complete text unit, not just the individual words.
  Sentence-BERT(SBERT) is a popular model specifically designed to produce high-quality sentence embeddings. It's built on the BERT architecture but is fine-tuned for semantic similarity tasks, making it highly effective at comparing the meaning of different sentences.
  OpenAI's text-embedding-ada-002 is a powerful general-purpose embedding model that creates a single vector for a given text input, which can be a sentence, paragraph, or document.
- Sentence Embeddings are a core component of RAG systems, they are used in the retrieval phase to find relevant information.
- **RAG Flow**
  1. **Document Chunking**: Large documents are broken down into smaller, meaningful chunks (e.g., paragraphs or sentences).
  2. **Embedding Generation**: Each chunk is converted into a sentence embedding using a model like SBERT.
  3. **Question Embedding**: User question is converted into a sentence embedding using the same model.
  4. **Similarity Search**: Vector database finds chunks with embeddings most similar to the question's embedding.
  5. **Answer Generation**: Retrieved relevant information is passed to an LLM to generate the final answer.
- **Embedding Model Selection**
  1. **General Purpose**: Models like OpenAI embeddings and Sentence-BERT are excellent for a wide range of tasks, including semantic search, clustering, and classification.
  2. **Domain-Specific**: BioBERT(Trained on biomedical literature for tasks in medical and life sciences domain), FinBERT(Specialized for financial news and documents).
  3. **Dimensionality**: This is the size of the embedding vector. Higher dimensions can capture more information, but increase computational cost and storage requirements.
  4. **Computational Cost**: Consider the resources required for both generating the embeddings and performing similarity searches. Larger models and higher dimensionality lead to greater costs in time and memory.
- **Vector Database**: To manage and search vector embeddings efficiently at scale. Embeddings are numerical representations of unstructured data, where the distance between vectors in a multi-dimensional space reflects the similarity of the original data.
- **Key Operations**: The primary operations are inserting new vectors, updating existing ones, and most importantly, similarity search. Many vector databases also support metadata filtering, allowing you to combine semantic search with traditional database queries.
- **Distance Metrics**: To measure how "close" two vectors are, vector databases use different mathematical metrics
  1. **Cosine Similarity**: Measures the angle between two vectors, focusing on their direction.
  2. **Euclidean Distance**: Measures the straight-line distance between two vectors, focusing on magnitude.
  3. **Dot Product**: A hybrid approach that combines direction and magnitude.
- Searching through millions or billions of vectors one by one (brute-force search) is too slow.
- Vector databases use **Approximate Nearest Neighbor (ANN)** algorithms to perform fast, efficient searches with a minimal trade-off in accuracy.
- **HNSW (Hierarchical Navigable Small World)**: A graph-based algorithm that creates multiple layers of interconnected nodes. It starts the search on the top layer (with few nodes) and moves down to the densest layer, quickly narrowing down the search space to find the nearest neighbors. It's known for its excellent balance of speed and accuracy.

- **IVF (Inverted File)**: A clustering-based approach. It partitions the vectors into clusters. When a query comes in, the search is limited to a few nearby clusters, rather than the entire dataset.
- **LSH (Locality-Sensitive Hashing)**: A hash-based method that "hashes" similar vectors into the same buckets, allowing for quick retrieval. It's often used for large-scale, low-accuracy searches.
- **Chroma**: A lightweight, open-source, and Python-native vector database.
  - Simple and ease of use, making it ideal for prototyping, local development and small-scale applications.
  - It can run in embedded mode, meaning it operates within application's memory without a separate server, or in client-server mode for more flexibility.
  - A great choice for getting started with RAG or other vector-based projects.
  - `https://github.com/chroma-core/chroma`
- **Pinecone**: A fully managed cloud service
  - Enterprise-grade performance, scalability, and ease of use at a high level.
  - Don't have to worry about infrastructure, maintenance, or scaling.
  - Automatically handles large datasets and high query loads, making it the preferred choice for production applications and large-scale deployments where performance and reliability are critical. `https://www.pinecone.io/`
- **Weavite**: Another prominent vector database that is open-source but also offers a managed cloud service.
  - Stands out for its built-in machine learning models and sophisticated data modeling.
  - It features a GraphQL API for easy querying and supports hybrid search, which combines vector-based semantic search with traditional keyword-based search.
  - Well-suited for applications that require complex data relationships and a mix of different searches.
  - `https://weaviate.io/`

Category	Database / Library	Key Characteristics	Best For
Dedicated Vector Databases	**Pinecone**, **Milvus**, **Weaviate**, **Qdrant**, **Chroma**	Built from the ground up for high-dimensional vector data. Offer full database features like data management, security, and scalability. Many provide both open-source and managed cloud options.	Production applications, large-scale deployment, and projects needing a full-featured, managed service.
Vector Search Libraries	**FAISS**, **ScaNN**, **Annoy**	Lightweight libraries focused on the core approximate nearest neighbor (ANN) algorithms. They don't have full database features like CRUD operations or horizontal scaling built-in, but are highly optimized for speed.	Research, in-memory search, and building custom solutions where you want full control over the underlying indexing and search process.
Traditional Databases with Vector Capabilities	**pgvector** (PostgreSQL), **Elasticsearch**, **Redis**, **MongoDB**	Existing databases that have added vector search as a feature, often through extensions or new data types. They are great for unifying structured and unstructured data in a single system.	Projects where you already use the database and want to add vector search without managing a separate system.

Figure 10: Comparison by Type

- **Development Phase**: For rapid prototyping, local testing, and learning, Chroma is the go-to due to its simplicity and zero-setup nature.
- **Production Scale**
  - For high-traffic, mission-critical applications, choose a production-ready solution.
  - Pinecone is excellent if you want a fully managed service and don't want to deal with infrastructure.
  - Weaviate is a good alternative if you need more control, require hybrid search, or prefer an open-source solution with a flexible architecture.

- **Cost**: Managed services like Pinecone have a subscription-based pricing model, while open-source solutions like Chroma and Weaviate (self-hosted) have a lower direct cost but require you to manage your own infrastructure.

- **Data splitting**, or **chunking**, is the process of breaking down large documents into smaller, more manageable pieces before they are indexed for a RAG system.

- Chunking is essential for three main reasons

  1. **Context Window Constraints**: LLMs have a limited context window, which is the maximum number of tokens they can process at one time. If a retrieved document is too large, the LLM won't be able to "see" the entire content, and key information might be lost.
  2. **Retrieval Precision**: A smaller, more focused chunk is more likely to be highly relevant to a user's query. If a document is indexed as one large chunk, a query might match only a small part of it.
  3. **Processing Efficiency**: Smaller chunks are easier and faster to process during both the embedding and retrieval phases. They reduce computational load and speed up the entire RAG pipeline.

- **Fixed-Size Chunking**: Documents are split into chunks of a predefined size, measured by character or token count (e.g., 500 tokens).
  Pros: It's easy to implement, predictable, and fast to process.
  Cons: This method is "unintelligent." It can cut a sentence or a paragraph in half, potentially breaking a semantic unit and causing information loss.

- **Semantic Chunking**: It uses NLP rules or structural cues (like headings) to split documents at meaningful points, such as the end of a sentence or a paragraph.
  Pros: It preserves the integrity of complete thoughts and ideas, leading to higher-quality retrieval.
  Cons: It's more complex to implement and can result in chunks of varying sizes, which might not be ideal for some systems.

- **Overlapping Windows**: Often used in conjunction with fixed-size chunking. It involves creating chunks that intentionally overlap with their neighbors. For example, a 500-token chunk might overlap with the next chunk by 100 tokens.
  Purpose: The primary goal is to prevent the loss of critical information that might be split between two chunks. The overlap provides a "bridge" of context.
  Trade-off: This increases the total number of chunks, which requires more storage and can slightly increase retrieval time. However, the improved quality of the retrieved information often makes this a worthwhile trade-off.

- **Hierarchical Windows**: An advanced strategy that creates chunks at multiple levels of granularity. For a long document, you might create top-level chunks for each major section, mid-level chunks for paragraphs within those sections, and low-level chunks for individual sentences.
  Method: The system creates a parent-child relationship between the chunks. The parent chunk might be a high-level summary, while the child chunks contain the detailed information.
  Use Case: This is ideal for complex, multi-layered documents like legal contracts or technical manuals. It allows the system to first retrieve the most relevant high-level section and then drill down to find the specific, detailed information.

- **Batch Processing**: To speed up the process, generate embeddings in large batches. Instead of sending one chunk at a time, group hundreds or thousands of chunks together. This is much more efficient as it reduces communication overhead.

- **Normalization**: If you're using cosine similarity for retrieval, it's a best practice to normalize your vectors. Normalizing ensures that all vectors have a length of 1, so the similarity calculation is based purely on the direction of the vectors, not their magnitude.

- **GraphRAG**: Combines knowledge graphs + vector retrieval. Enables multi-hop reasoning through relationships. Better for complex queries requiring entity connections.

- **RAGAS (Retrieval-Augmented Generation Assessment)** is a framework designed to evaluate the quality of a RAG pipeline. It uses metrics specifically tailored to the RAG process, going beyond traditional metrics like BLEU or ROUGE, which only assess the final answer's text against a reference answer.

- **Multimodal RAG**: Process visual + audio + text content. Multi-modal embeddings (CLIP, ImageBind). Timestamp-aware retrieval with video references

5 Finetuning

6 Alignment Methods

7 AI Agents

8 AI Safety

9 LLMOps

10 Federated Learning